Process Verification

Final Project

Quentin Meneghini

20th May 2026

Facultat d'Informàtica de Barcelona

Contents

Chapter 1

Design Overview

1.1 Overview

The `gcd` module computes the Greatest Common Divisor of two unsigned integers using the subtractive Euclidean algorithm. It processes one input pair at a time and communicates through two independent ready/valid handshake channels.

Mathematical Definition

For unsigned operands A and B:

$$\gcd(A, 0) = A$$

$$\gcd(0, B) = B$$

$$\gcd(0, 0) = 0 \quad \text{(covers both above)}$$

$$\gcd(A, A) = A$$

$$\gcd(A, B) = \gcd(A - B, B) \quad \text{when } A > B$$

$$\gcd(A, B) = \gcd(A, B - A) \quad \text{when } B > A$$

Operation Sequence

1. **IDLE:** The module waits in the `IDLE` state, asserting `in_ready = 1`.
2. **Handshake & RUN:** On the input handshake, operands are latched. The result is determined immediately for zero or equal operands; otherwise, iterative subtraction begins

in the RUN state.

3. **DONE:** When convergence is reached, the result is latched into the output register and the module enters the DONE state, asserting out_valid = 1.

4. **Return:** After the output handshake completes, the module returns to IDLE.

1.2 Parameters

Parameter	Default	Description
WIDTH	16 (must be ≥ 1)	Width of the data operands

Table 1.1: Module Parameters

1.3 Top-Level Interface

Clock and Reset

Signal	Direction	Width	Description
clk	Input	1	Clock signal
rst_n	Input	1	Active-low sync reset

Input Channel

Signal	Direction	Width	Description
in_valid	Input	1	Asserted by the producer when a_in and b_in hold valid data
in_ready	Output	1	Asserted by the module when it can accept a new transaction. High only in the IDLE state.
a_in	Input	WIDTH	First operand
b_in	Input	WIDTH	Second operand

Output Channel

Signal	Direction	Width	Description
out_valid	Output	1	Asserted by the module when gcd_out holds a valid result. High for the entire DONE state.
out_ready	Input	1	Asserted by the consumer when it is ready to accept the result.
gcd_out	Output	WIDTH	GCD result. Valid and stable for the entire period that out_valid is asserted.

1.4 Functional Description

The behavior of the `gcd` module is driven by a Finite State Machine (FSM) comprising three states:

FSM States

State	in_ready	out_valid	Description
IDLE	1	0	Waiting for input. Ready to accept a new transaction.
RUN	0	0	Iterative subtraction in progress. No new input accepted.
DONE	0	1	Result ready. Waiting for the consumer to accept it.

Algorithm Execution (RUN State)

Each clock cycle while in the RUN state, the working registers are updated combinatorially for the next cycle:

- **If** `a_reg > b_reg`: `a_next = a_reg - b_reg` and `b_next = b_reg`.
- **Else if** `b_reg > a_reg`: `b_next = b_reg - a_reg` and `a_next = a_reg`.

State Transitions

IDLE → DONE: Triggered by an input handshake **AND** (`a_in == 0 OR b_in == 0 OR a_in == b_in`). The result is known immediately, making this a 1-cycle transition.

IDLE → RUN: Triggered by an input handshake **AND** (`a_in != 0 AND b_in != 0 AND a_in != b_in`).

RUN → DONE: Triggered when the next-cycle values of the working registers are equal (`a_next == b_next`). This evaluates against the combinatorial *next-cycle* values, firing on the same clock edge as the final subtraction step.

DONE → IDLE: Triggered by a successful output handshake (`out_valid == 1 AND out_ready == 1`).

Appendix A

Appendix Chapter

A.1 Use of tools

- Visual Studio Code: systemverilog IDE.
- QuestaSim: CLI tester.
- GTKwave: wave visualizer.

A.2 Use of time

A total of h distributed as follows.

- Understanding the problem: 1h.
- Report: 1h.

A.3 Use of AI

- Gemini (Pro): sentence enhancing, bug fixer